

DEEP LEARNING

Professor: Horacio Larreguy

TAs: Eduardo Zago, Ernesto Dávila, Jose Luis Perez

ITAM Applied Research 2 / Economics Research
Seminar,
11/01/2023

GENERAL PERSPECTIVE

INTRODUCTION TO DEEP LEARNING

PERCEPTRON

GRADIENT DESCENT

BACK-PROPAGATION

ACTIVATION FUNCTIONS

REGULARIZERS

NEURAL NET EXAMPLE USING TENSORFLOW

WHAT IS DEEP LEARNING?

- ▶ Subset of ML, involves using **Neural Networks** to learn from data.
- ▶ **Difference from ML:** more complex models with multiple layers that can learn from raw data.
- ▶ Reduces the need for manual feature engineering.
- ▶ They have been found to perform better than ML models in complex tasks such as speech recognition, NLP, Image analysis and robotics.

NEURAL NETWORKS

- ▶ Inspired by the structure of the brain.
- ▶ Series of interconnected processing nodes which work together to perform complex computations.
- ▶ Three Types of Layers:
 1. **Input layer:** receives the raw data
 2. **Hidden layer:** perform a series of mathematical operations on the input data to extract meaningful features and patterns
 3. **Output layer:** makes the prediction or classification.

NEURAL NETWORK

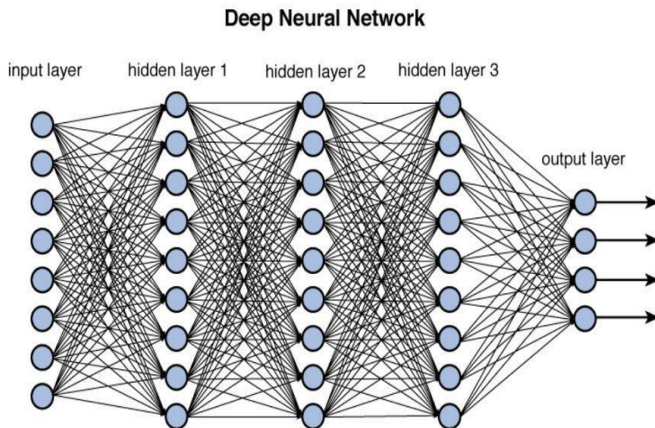
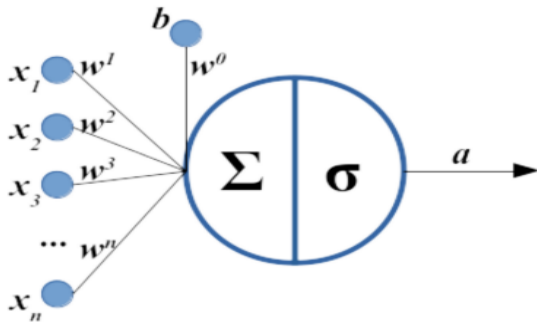


Figure 12.2 Deep network architecture with multiple layers.

PERCEPTRON

- ▶ To better understand the structure of a NN let us look at the simplest one: the **perceptron**.
- ▶ A **perceptron** is a single neuron (node) connected to one or more input nodes, with a weight associated to each.
- ▶ The neuron sums up the weighted inputs and applies an **activation function** to produce an output.
- ▶ Let us look at one graphically and mathematically:

PERCEPTRON, GRAPHICALLY



PERCEPTRON, MATHEMATICALLY

- Mathematically, if we have a Sigmoid¹ activation function $\sigma(s)$, then:

$$\begin{aligned} s &= w^T x, \\ &= \sum_{n=0}^N w_n x_n, \\ &= \sum_{n=1}^N w_n x_n + w_0 n_0 \end{aligned} \tag{1}$$

And

$$a = \sigma(s)$$

¹Similar to what the Logistic function does in a Logit model.

PERCEPTRON, MATHEMATICALLY

- ▶ $a = \sigma(s)$ would be the output signal, or prediction, that takes the weighted sum of inputs s and applies a non-linearity to them.
- ▶ The immediate question that follows from this is: *how can we estimate the parameter w_i ?*
- ▶ We'll use an iterative minimization algorithm called **Gradient Descent** to update our parameters gradually.

GRADIENT DESCENT

- ▶ Before explaining Gradient Descent we must remember a few things:
- ▶ We are using a **data-driven approach**, thus we have a labeled data set.
- ▶ We defined the loss as the difference between the **predicted** output and the **actual** output.
- ▶ For simplicity:

$$E = (y - \hat{y})^2$$

- ▶ Recall that we **minimize** the objective function of a given problem by moving our parameters in the opposite direction of the gradient.

GRADIENT DESCENT

- Therefore, since our parameters are w_i and our objective function (the one we want to minimize) is E , we have that:

$$w^{i+1} = w^i - \nu \frac{\partial E}{\partial w^i}$$

- In our context, since:

$$\begin{aligned} s &= w^T x, \\ \hat{y} &= \sigma(s) \\ E &= (y - \hat{y})^2 \\ \sigma'(s) &= \sigma(s)(1 - \sigma(s)) \end{aligned} \tag{2}$$

GRADIENT DESCENT

- We would have that:

$$\begin{aligned}\frac{\partial E}{\partial w_n^i} &= \frac{\partial (y - \hat{y})^2}{\partial w_n^i}, \\ &= -2(y - \hat{y})\sigma(s)(1 - \sigma(s))x_n\end{aligned}\tag{3}$$

Therefore,

$$w_n^{i+1} = w_n^i + 2\nu(y - \hat{y})\sigma(s)(1 - \sigma(s))x_n$$

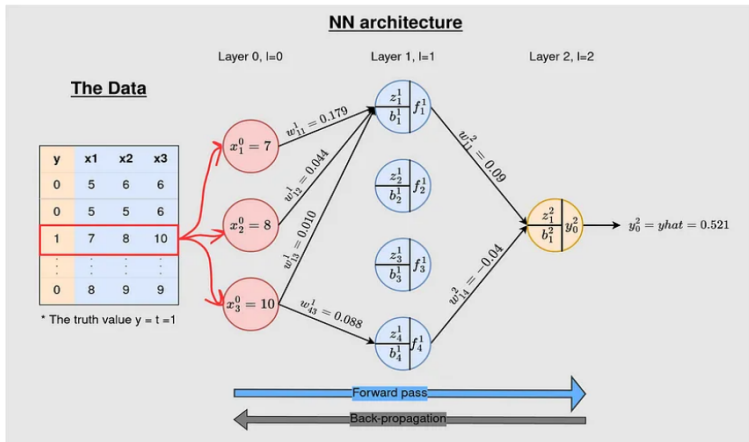
GRADIENT DESCENT, ALGORITHM

► Thus, the training process for batch i at epoch e , is given by:

1. Random initialization of weights $\implies w_0$
2. Forward pass $\implies \hat{y} = f(x; w)$
3. Error estimation $\implies E(y, \hat{y})$
4. Gradient computation $\implies \frac{\partial E}{\partial w_n}$
5. Backward pass (weight adjustment) $\implies w_{n+1} = w_n - \frac{\partial E}{\partial w_n}$

INFORMATION FLOW

- Now, how do we do this for more than one neuron and several hidden layers?



BACK-PROPAGATION

- ▶ **Back-propagation** is a method for supervised learning used by NN to update parameters.
- ▶ For Neural Nets with multiple layers, the backward pass can get really complicated, so we must understand how the information flows.
- ▶ We must do the backward pass (weight adjustment) of all weights in the model.
- ▶ Some of these weights are concatenated inside several activation functions.
- ▶ Thus, to obtain the derivative, we must calculate the **Chain Rule**.

BACK-PROP, CHAIN RULE

Recall that the chain rule is used to differentiate any function of the form:

$$h(x) = f(g(x))$$

And the derivative is given by:

$$h'(x) = f'(g(x))g'(x)$$

In a more general context, if we have a function:

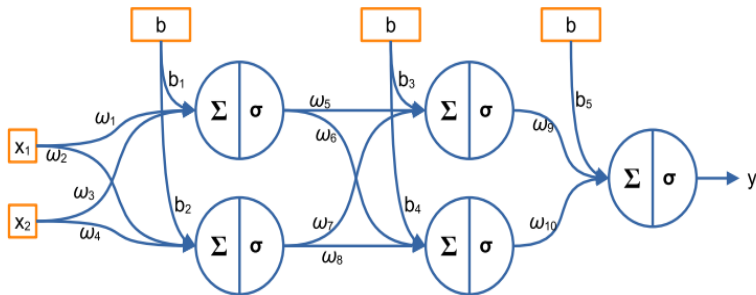
$$f_{1..n} = f_1(f_2(f_3(\dots f_{n-1}(f_n(x))\dots)))$$

The derivative would be given by:

$$f'_{1..n} = f'_1(f_{2..n}(x)) \cdot f'_2(f_{3..n}(x)) \cdot \dots \cdot f'_{n-1}(f_{n..n}(x)) \cdot f'_n(x)$$

BACK-PROP, EXAMPLE

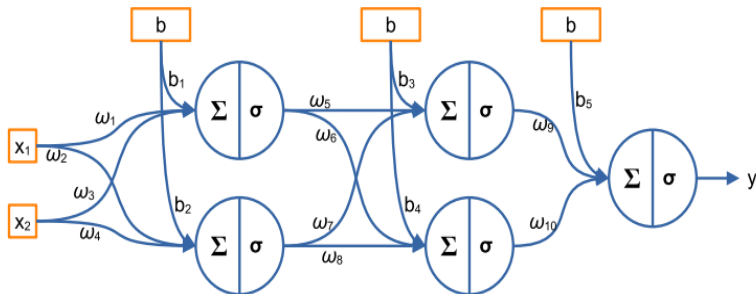
- ▶ As an example, let us look at the next NN:



- ▶ Also recall that we defined:

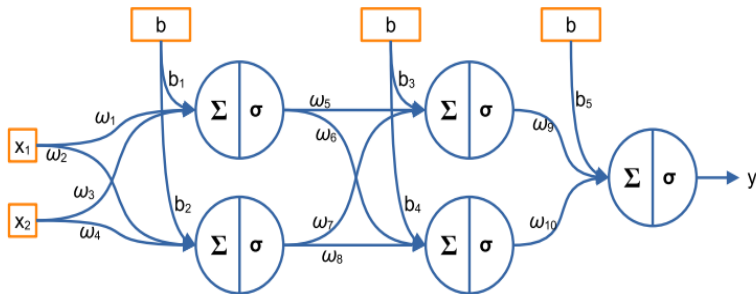
$$\begin{aligned} s &= w^T x, \\ a &= \sigma(s) \end{aligned} \tag{4}$$

WIDE NEURAL NET



- Notice we have: 5 activation functions (2 for each hidden layer and one in the final layer)
- Also, at each node there is a linear combination of the weights and the input data (for the first layer)
- And a linear combination of the output of the last layer and their respective weights.

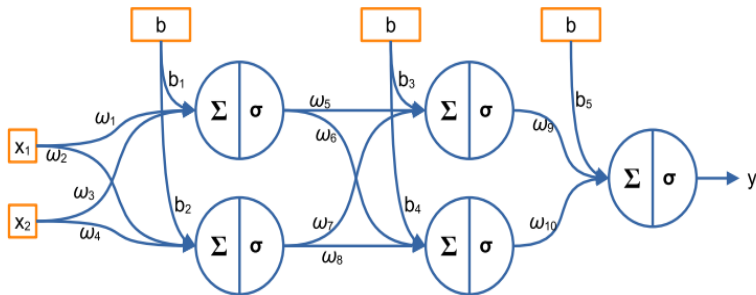
WIDE NEURAL NET



- The linear combination at node 1 and 3 (for example) would be given by:

$$\begin{aligned} s_1 &= b_1 + w_1 \cdot x_1 + w_3 \cdot x_2, \\ s_3 &= b_3 + w_5 \cdot a_1 + w_7 \cdot a_2 \end{aligned} \tag{5}$$

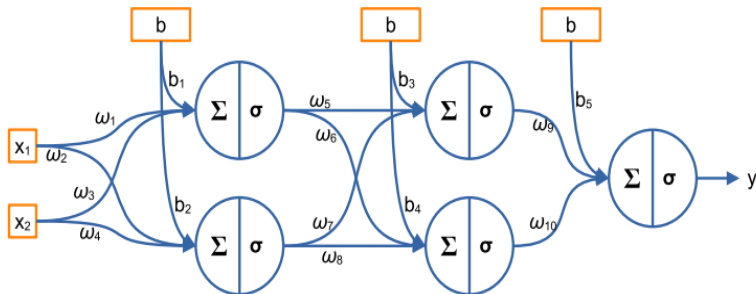
BACK-PROP, TAKING DERIVATIVES



► Thus, the derivative of E with respect to w_i is:

$$\begin{aligned} \frac{\partial E}{\partial w_1} = & \frac{\partial E}{\partial a_5} \frac{\partial a_5}{\partial s_5} \frac{\partial s_5}{\partial a_3} \frac{\partial a_3}{\partial s_3} \frac{\partial s_3}{\partial a_1} \frac{\partial a_1}{\partial s_1} \frac{\partial s_1}{\partial w_1}, \\ & + \frac{\partial E}{\partial a_5} \frac{\partial a_5}{\partial s_5} \frac{\partial s_5}{\partial a_4} \frac{\partial a_4}{\partial s_4} \frac{\partial s_4}{\partial a_1} \frac{\partial a_1}{\partial s_1} \frac{\partial s_1}{\partial w_1} \end{aligned} \quad (6)$$

BACK-PROP



► Why the sum of products?

$$\begin{aligned} \frac{\partial E}{\partial w_1} = & \frac{\partial E}{\partial a_5} \frac{\partial a_5}{\partial s_5} \frac{\partial s_5}{\partial a_3} \frac{\partial a_3}{\partial s_3} \frac{\partial s_3}{\partial a_1} \frac{\partial a_1}{\partial s_1} \frac{\partial s_1}{\partial w_1}, \\ & + \frac{\partial E}{\partial a_5} \frac{\partial a_5}{\partial s_5} \frac{\partial s_5}{\partial a_4} \frac{\partial a_4}{\partial s_4} \frac{\partial s_4}{\partial a_1} \frac{\partial a_1}{\partial s_1} \frac{\partial s_1}{\partial w_1} \end{aligned} \quad (7)$$

WEIGHT UPDATING

- ▶ Finally, as seen in the last chapter, weight 1 gets updated in the following form:

$$w_1^{i+1} = w_1^i - \frac{\partial E}{\partial w_1^i}$$

- ▶ And this happens to all the weights at each time of training t (batch or epoch).
- ▶ Notice that the impact of back-prop is proportional to the depth of the layer.
- ▶ Weights in shallow layers are updated more softly with respect to those in deeper layers.

ACTIVATION FUNCTIONS

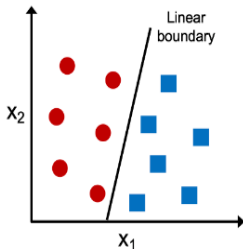
- ▶ An **activation function** defines how the weighted sum of the input is transformed into an output from a node (or nodes) in a layer of a network.
- ▶ Introduces a **non-linearity** into the output of the neuron.
- ▶ Allows the neural network to learn complex patterns and relationships in the data.
- ▶ In the **hidden layers**, the activation function will control how well the model learns.
- ▶ In the **output layer**, the activation function will define the type of predictions the model can make.

LINEAR VS. NON-LINEAR PROBLEMS

- ▶ Activation functions allow the Neural Net to perform well in non-linear separable problems²:

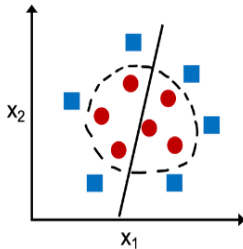
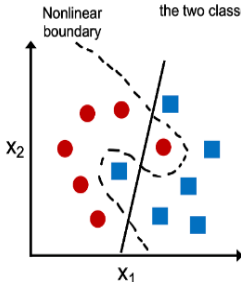
Linearly separable

A linear decision boundary that separates the two classes exists



Not linearly separable

No linear decision boundary that separates the two classes perfectly exists



²Image taken from [VitalFlux](#)

LINEAR AND RELU ACTIVATIONS

- ▶ **Linear:** used at the output layer for unbounded regression, but also works for binary classification.

$$a = \sum_{n=0}^N x_n w_n, \quad \frac{\partial a}{\partial w_i} = x_i$$

- ▶ **ReLU:** maps its input to the maximum of 0 and the input value. Very efficient and works well in tons of contexts.

$$a = \max(0, x)$$

SIGMOID AND TANH ACTIVATIONS

- **Sigmoid:** mostly used at the output layer for binary classification problems and regression with $0 \leq y \leq 1$.

$$a = \sigma(s) = \frac{1}{1 + e^{-s}}, \quad \sigma'(s) = \sigma(s)(1 - \sigma(s))$$

- **Tanh:** Better than sigmoid in hidden layers since it resembles the identity function more closely, $\tanh(0) = 0$.

$$a = \frac{e^s - e^{-s}}{e^s + e^{-s}}, \quad a' = 1 - a^2$$

SOFTMAX

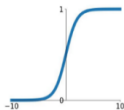
- ▶ Used for multi-class classification
- ▶ Used to exaggerate the most probable of the elements of the vector.
- ▶ It maps its input to a probability distribution over the output classes, allowing the neural network to produce a probability score for each class.

$$a_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \quad a'_i = a_i(1 - a_i)$$

ACTIVATION FUNCTIONS GRAPHICALLY

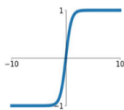
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



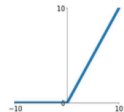
tanh

$$\tanh(x)$$



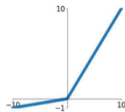
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

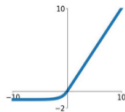


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



REGULARIZERS

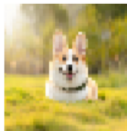
- ▶ Recall that regularizers are techniques used to reduce **over-fitting** sometimes at the expense of increased **under-fitting**.
- ▶ In a ML context we went over two techniques: L1 and L2 regularization.
- ▶ They both could be used in DL models, as we have an objective function (the Error).
- ▶ Other techniques used are: **Dataset Augmentation**, **Early Stopping** and **DropOut**.

DATASET AUGMENTATION

- ▶ This technique consists of increasing the dataset by creating **fake data** and adding it to the training set.
- ▶ Usually used in classification tasks to increase the model's robustness.
- ▶ Prediction needs to be invariant to a wide variety of transformations.
- ▶ A good example of a context where it can be used is in object recognition, since you can rotate, change the color and scale images, to generate new data points.³

³Image taken from [DataCamp](#).

DATA AUGMENTATION



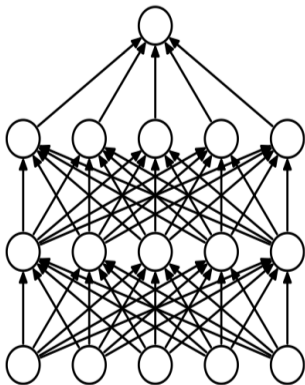
EARLY STOPPING

- ▶ Refers to returning to the parameters, epoch, where the validation accuracy last improved, and use them as if they were the last parameters.
- ▶ Or, stopping the algorithm when no parameters have improved over the best recorded validation error.
- ▶ Very unobtrusive technique.
- ▶ Could be seen as a restriction to the parameters to a neighborhood near w^0 .

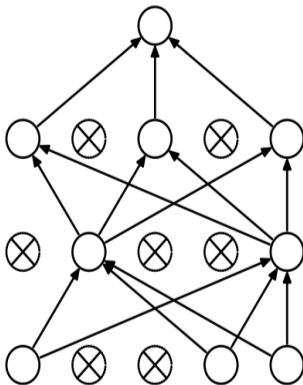
DROP OUT

- ▶ This technique limits the capacity of the model at random by deactivating neurons (dropout) or weights (drop-connect).
- ▶ We must choose the dropout probability, which becomes then another hyper-parameter.
- ▶ Very popular technique since it is very computationally cheap (in contrast to L1 or L2)
- ▶ Helps the network be robust against variations.

DROP OUT



(a) Standard Neural Net



(b) After applying dropout.

NEURAL NET EXAMPLE USING TENSORFLOW

- ▶ **Tensorflow** is an end-to-end open source platform for machine learning.
- ▶ **TensorFlow**'s high-level APIs are based on the **Keras** API standard for defining and training neural networks.
- ▶ **Keras** enables fast prototyping, state-of-the-art research, and production—all with user-friendly APIs.
- ▶ Let's install them:

```
!pip install keras  
!pip install tensorflow
```

LOAD THE PACKAGES AND DEFINE THE PROBLEM

- ▶ Let's first load the packages and the data set:

```
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

from sklearn.datasets import load_iris
iris = load_iris()
```

- ▶ We will again use the Iris Data Set, but we will increase the difficulty of the problem by a lot.
- ▶ We will try to **predict the length and width of sepals using the length and width of the petals.**

PREPARE THE DATA

- ▶ As always, let's prepare our data by splitting the sample into training and test:

```
X = iris.data[:, :2]
Y = iris.data[:, 2:]
L = iris.target

# Train Test Split:
from sklearn.model_selection import train_test_split
x_train, x_test, y_train,
    y_test, l_train, l_test = train_test_split(X, Y, L,
        test_size=0.2, random_state = 42)
```

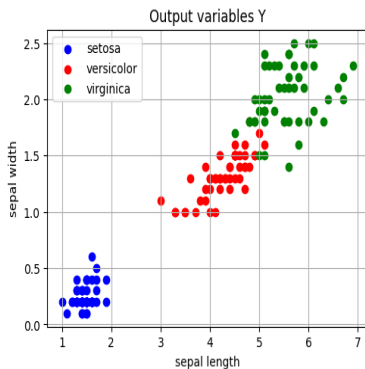
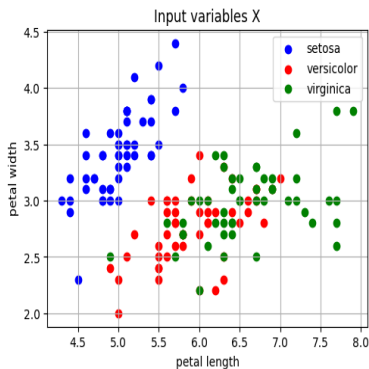
VISUALIZING THE DATA

- We can visualize the problem by graphing each set of features and their classes:

```
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.scatter(X[L==0, 0], X[L==0, 1], c='b', label='setosa')
plt.scatter(X[L==1, 0], X[L==1, 1], c='r', label='versicolor')
plt.scatter(X[L==2, 0], X[L==2, 1], c='g', label='virginica')
plt.legend()
plt.grid(True)
plt.xlabel('petal length')
plt.ylabel('petal width')
plt.title('Input variables X')
plt.subplot(1, 2, 2)
plt.scatter(Y[L==0, 0], Y[L==0, 1], c='b', label='setosa')
plt.scatter(Y[L==1, 0], Y[L==1, 1], c='r', label='versicolor')
plt.scatter(Y[L==2, 0], Y[L==2, 1], c='g', label='virginica')
plt.legend()
plt.grid(True)
plt.xlabel('sepal length')
plt.ylabel('sepal width')
plt.title('Output variables Y')
plt.show()
```

VISUALIZING THE DATA

- It is indeed a more complex problem, since we are estimating a function that goes from $\mathbb{R}^2$ to $\mathbb{R}^2$.



DEFINING THE MODEL

- ▶ Let's start defining our model:
- ▶ We have two inputs, thus the input layer takes a shape of (2,):

```
i = Input(shape=(2,), name='input')
```

- ▶ We will define a model with **3 hidden layers**, 32 neurons in the first and the third, and 64 in the 2nd one.
- ▶ For activations we will use ReLu for the hidden layer and Linear in the final:

```
h = Dense(units=32, activation='relu', name='hidden1')(i)
h = Dense(units=64, activation='relu', name='hidden2')(h)
h = Dense(units=32, activation='relu', name='hidden3')(h)
o = Dense(units=2, activation='linear', name='output')(h)
```

DEFINING THE MODEL

- ▶ Finally, we aggregate our model using the `Model()` function and obtain an overall description of it:

```
MLP = Model(inputs=i, outputs=o)
MLP.summary()
```

```
Model: "model"
```

Layer (type)	Output Shape	Param #
input (InputLayer)	[(None, 2)]	0
hidden1 (Dense)	(None, 32)	96
hidden2 (Dense)	(None, 64)	2112
hidden3 (Dense)	(None, 32)	2080
output (Dense)	(None, 2)	66

```
=====  
Total params: 4,354  
Trainable params: 4,354  
Non-trainable params: 0  
=====
```

COMPILING THE MODEL

- ▶ To start training the model, we first need to choose another set of hyper-parameters: the optimization procedure and the error function.
- ▶ We will use the ones we have seen, **Stochastic Gradient Descent (SGD)** for optimization and **Mean Square Error (MSE)** for Error:

```
# Compile it  
MLP.compile(optimizer='sgd', loss='mse')
```

TRAINING THE MODEL

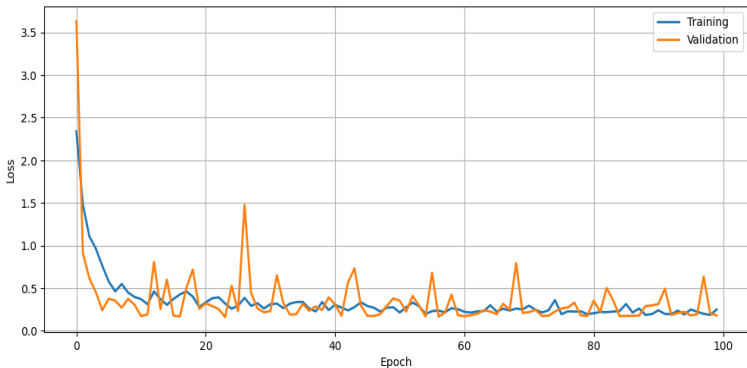
- ▶ And we can start training it using our data sets. We choose the last two hyper-parameters: batch size and epochs.

```
# Train it  
MLP.fit(x=x_train, y=y_train, batch_size=4,  
        epochs=100, verbose=2, validation_split=0.1)
```

- ▶ We have 100 values for the loss and the accuracy, both in training in validation.
- ▶ To visualize them we will use the Learning Curve we saw in the ML lecture.

VISUALIZING METRICS

```
plt.figure(figsize=(12, 5))
plt.plot(MLP.history.history['loss'], label='Training', linewidth=2)
plt.plot(MLP.history.history['val_loss'], label='Validation', linewidth=2)
plt.legend()
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.show()
```



MODEL'S PERFORMANCE

- ▶ And we can conclude that the model is not under-fitted, since the loss is really low in training.
- ▶ We also notice that the training loss and the validation loss are really similar at all epochs, therefore the model is not over-fitted.

```
# Evaluate on the test set  
MLP.evaluate(x=x_test, y=y_test, verbose=False)  
  
### Output: 0.10164663195610046
```

- ▶ We obtained an average loss of 10%, which given that this is a regression problem, is very good.